THE JAM_BOT, A REAL-TIME SYSTEM FOR COLLABORATIVE FREE IMPROVISATION WITH MUSIC LANGUAGE MODELS

Lancelot Blanchard[1,*] **Perry Naseck**[1,*] **Stephen Brade**[2]
Kimaya Lecamwasam[1] **Jordan Rudess**[1,3,♯] **Cheng-Zhi Anna Huang**[2]
Joseph Paradiso[1]

[1] MIT Media Lab, Cambridge, MA, USA
[2] MIT Music Tech, Cambridge, MA, USA
[3] Wizdom Music, New City, NY, USA
[*] Authors contributed equally
[♯] Work performed as part of a Visiting Artist Residency
lancelot@media.mit.edu, pnaseck@media.mit.edu

ABSTRACT

In order to design a Generative AI system that could improvise on stage with GRAMMY-winning keyboard virtuoso Jordan Rudess, we developed the "JAM_BOT", a real-time performance system that could match his eclectic improvisational aesthetics. We debuted the JAM_BOT at a high-stakes sold-out concert to critical acclaim, realizing a series of virtuosic tightly-coupled Human-AI free improvisations in varying musical styles. Reflecting on our year-long collaboration, we summarize learnings for AI researchers and musicians on the adaptations needed to turn state-of-the-art symbolic music Language Models (LMs) into JAM_BOTS and the engineering required to make them performance-ready. We focus on three aspects: (1) enabling JAM_BOTS to take on different musical roles by adapting music LMs to employ different interaction strategies by modifying the context and conditioning signals; (2) describing how Rudess intentionally structures his improvisation in order to finetune JAM_BOTS to match the style needed for each piece; and (3) showing the optimizations needed to run music LMs in real-time and embed them in a low-latency multi-threaded system that listens, prompts, and schedules model generations seamlessly. We hope these insights enable more musician-AI symbiotic virtuosity.

1. INTRODUCTION

On September 21, 2024 at the MIT Media Lab, we put our Generative AI–powered JAM_BOT to the test together with Jordan Rudess in a high-stakes sold-out concert. Jordan Rudess is known for his versatility as an improviser and his eclectic musical performances which showcase his virtuosity in distinct genres. When planning for this performance, Rudess wanted to improvise with an AI system on stage in real-time instead of working with Generative AI off-stage in an offline fashion. Through this system, Rudess wanted to be able to "improvise with [himself]" and train models that could understand "the language and logic behind the way [he] improvised" to "push boundaries", which would enable him to attain a new form of "*symbiotic virtuosity*" together with the system [1]. As such, we designed a Generative AI system that could improvise on stage with him through a range of musical pieces of varying styles. In this paper, we reflect on this year-long close collaboration and explore how Rudess's unique aesthetic approaches to improvisation shaped the design of a novel real-time performance system, which we call the JAM_BOT.

We situate our work in the context of *free improvisation*, where musicians perform without a predefined musical score or structure. With the integration of a JAM_BOT, how can the artist dynamically choreograph the piece? How will they and the JAM_BOT *coordinate* what and when to play? How can they anticipate *what to expect* from the JAM_BOT, and maintain sufficient *agency* and *controllability* to guide the improvisation, especially when everything must proceed smoothly during a high-stakes performance?

To tackle these challenges, we collaborated closely with Jordan Rudess to explore how autoregressive symbolic music Language Models (LMs) could be adapted into JAM_BOTS, enabling virtuosic, tightly-coupled Human-AI free improvisation. Although music LMs–from Music Transformer [2] to Anticipatory Music Transformers [3]–can generate compelling musical sequences, their outputs are often generic, their inference speeds too slow, and their input, output, and conditioning structures too rigid. To support highly-entangled Human-AI improvisations where musical role-switching is frequent, mechanisms for coordination or signaling are essential. The JAM_BOT also needs to pay "double attention" to both what the human musician

© L. Blanchard, P. Naseck, S. Brade, K. Lecamwasam, J. Rudess, C.Z.A. Huang, and J. Paradiso. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** L. Blanchard, P. Naseck, S. Brade, K. Lecamwasam, J. Rudess, C.Z.A. Huang, and J. Paradiso, "The JAM_BOT, a Real-Time System for Collaborative Free Improvisation with Music Language Models", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

is playing and its own past, present, and future [4].

Our contributions are threefold: First, we develop three interaction strategies for the JAM_BOT, enabling the system to take on different musical roles in the improvisation and the musician to shape the musical outcome. Second, for JAM_BOTS to match the style and identity needed for each piece, we describe how Rudess intentionally structures his improvisations in order to finetune music LMs to enable the aforementioned interaction strategies and model stylistic gestures. Third, we show the optimizations needed to run music LMs in real-time and how to embed them in a low-latency multi-threaded system that listens, and prompts and schedules model generations seamlessly.

The combination of these three contributions culminated in the 2024 concert. In this debut performance, Jordan Rudess improvised alongside the JAM_BOT across multiple musical pieces, each with distinct genres and identities, demonstrating that the system is not only able to listen and respond in real-time in a stylistic-specific fashion but also appeal to and engage a wide audience. We also release our code publicly in the hope that JAM_BOTS can be used in other performances.*

2. RELATED WORK

2.1 Symbolic Music Generation and Offline Interactions

Algorithmic composition is an ongoing curiosity of musicians and scientists that focuses on adapting computational advancements to the purposes of music creation. Early work explored Markov chains and genetic algorithms [5–7], while more recent research has introduced compelling techniques based on LSTM networks [8, 9], and later, Transformers [2,3]. Transformers, in particular, form the backbone of music LMs, which can generate music with local coherence and a compelling global arc. Offline interactive methods have emerged through unique adaptations of existing algorithms. For example, infilling techniques accommodate the nonlinear nature of human composition by training models to bridge gaps between existing musical arcs [10, 11]. Other strategies model multiple tracks of music [11–13] or condition symbolic generation on particular styles [14, 15]. Our work builds on these techniques by embedding state-of-the-art symbolic music generation into a real-time improvisation. We adapt the Anticipatory Music Transformer (AMT) [3] by developing performance-tested interaction strategies for controllable and compelling free improvisation.

2.2 Real-Time Musician-Machine Collaborations

Researchers have developed novel computational frameworks to enable real-time interactions between musicians and musical agents. Early systems established hard-coded links between musicians' gestures and synthetic performers [16], enabling simultaneous conducting and performance or semi-autonomous rule-based systems like Voyager [17]. Later works such as Continuator [18] used Markov chains to enable real-time responses that mirror a musician's improvisations and style. OMax Brothers [19] expands on this by using Markov models to create a modular real-time jam space for any number of musicians or synthetic performers. Markov models continue to appear in contemporary systems, including a recent live co-improvisation featured in WIRED.† In parallel, human-robot interaction research has embedded improvisational agents in anthropomorphic forms, lending interpretability and embodiment to co-improvisation [20–22], while fast audio models such as RAVE [23] can be leveraged for real-time timbre transfer in Max.‡ and has been deployed in interactive dance performances [24] Recent systems focused on symbolic music embed lightweight recurrent models to co-improvise in the style of Bach [25], or to map button presses to real-time piano output [26]. More recently, Transformers have been adapted for real-time use: ReaLChords and its companion, ReaLJam, perform adaptive live harmonization with strong robustness to unfamiliar melodic input, enabled by Reinforcement Learning [27, 28]. In this project, we specifically focus on symbolic music and contribute to this area by adapting a music LM to support three interactive paradigms for free musical dialogue with a virtuoso musician. We present our training pipeline for adapting AMT to these paradigms, and document our optimization of AMT for real-time performance.

3. DESIGNING & TRAINING THE JAM_BOT

During our iterative discussions with Jordan Rudess, we identified key design requirements for the JAM_BOT for optimal performability and to ensure Rudess's comfort on stage.

3.1 Modeling Separate Musical Identities

"I wanted to dig into specific parts of my musical personality and really explore what makes each one tick. Each model is like a deep dive into a different side of how I play or think musically, and building them separately gave me the freedom to shape them with real intention and nuance." (Jordan Rudess)

Jordan Rudess's performances typically consist of a collection of musical scenes, each showcasing his virtuosity in distinct genres. For the JAM_BOT to accompany his improvisation effectively, the system must independently understand these varying styles. Our experiments revealed that, in order to act as a convincing improvisation partner, the JAM_BOT needed not only to display *local coherence* with recent musical input, ensuring continuity, but also a strong *style adherence* to the artist's current improvisational style.

To ensure this *stylistic adherence*, we collect multiple small sets of training data that we use to train multiple models to use in the JAM_BOT system, each corresponding to the genre of a musical piece that Rudess can engage in. While state-of-the-art music LMs are very capable of generating long and coherent musical sequences, their

*Our code and resources can be accessed at https://jam-bot-ismir-2025.media.mit.edu/.

†https://www.wired.com/story/generative-ai-music/

‡https://github.com/acids-ircam/RAVE

large-scale training can sometimes be detrimental to their stylistic adherence and their output too generic. For example, AMT is trained on the Lakh dataset which has been demonstrated to be comprised of primarily electronic, pop, and classical music [29], biasing the style of the AMT's generation to these genres. Previous work [30] suggests that, in creative contexts, overfitting on small datasets can be a powerful mechanism for enabling greater human influence over Generative AI systems.

With this in mind, we recorded Rudess during his practice sessions and collected 15-45 minute-long MIDI clips. We augmented these data by transposing them to all twelve keys and used them to fine-tune a pretrained AMT model (`stanford-crfm/music-medium-800k`, approx. 360M parameters). Models were trained for 2,000 steps, with overfitting usually occurring as early as 300 steps in, where validation loss would start to plateau. While we do not currently plan to release the dataset, further statistical analyses can be found on our website.

3.2 Implementing Controllability

"I wanted the JAM_BOT *to feel like a version of myself–like if I could put my musical brain into another player and see what it would be like to jam with me. As someone who improvises by ear, that was a really exciting idea to explore. [...] [The system could] improvise and perform live in a duet with me. Sometimes leading, sometimes following, the model and I could create new and unique music."* (Jordan Rudess)

A key requirement for the JAM_BOT was its *controllability*. During free improvisation, the absence of predetermined structure and planning can make using an unpredictable generative system daunting. As such, Rudess required a mechanism to coordinate musical roles and provide *musical guidance* on stage that either *lead* or *followed* musical decisions and transition seamlessly between these roles, as well as accommodate *harmonic*, *melodic*, and *rhythmic cues*, with varying degrees of rigidity.

4. DEVELOPING INTERACTION STRATEGIES FOR THE JAM_BOT

Implementing these requirements, however, is challenging. In real-time settings, it is difficult to determine the optimal timing and context to simulate organic musical dialogue in autoregressive models. The inherent speed and complexity of Rudess's music specifically makes it impossible to naively prompt our model continuously, since the prompting rate needed to generate coherent sequences would be too high. As such, we took inspiration from human improvisers and the balance between their own coherent musical performance and their continuous focus on the musical information from fellow players to carefully decide when and how to best prompt the music LM. Norgaard et al. describe this dual-process phenomenon in expert jazz musicians as a conscious focus on higher-level musical elements and ensemble interaction, alongside a subconscious process for generating note choices [4]. To model this phenomenon, we develop three *interaction strategies* that balance the model's attention to external information and focus on its own composition (Fig. 1). Additionally, to ensure that the system displays strong *style adherence*, we craft precise training datasets for each interaction strategy.

4.1 At regular time intervals (1)

When & How to prompt The most straightforward approach to prompt the model is to do so at *regular time intervals* using the harmonic, melodic, and rhythmic content most recently played by the performer. Drawing from common practices in improvised music, this enables the human and the system to alternate ("trade") in their improvisation, with each yielding to the other after a predetermined time interval (e.g., 4 bars, 2 bars, 1 bar). During the system's turn, it focuses solely on its own output, while it listens to the human input when it is not playing.

Training Data & Performance To fine-tune a music LM to enable this kind of interaction, we collect long, single-instrument MIDI clips. Through this fine-tuning, the model is able to learn how to continue any sequence of musical input in a specific style, making it a good fit for this type of prompting strategy. In the final performance, Rudess was able to use this mechanism to create two pieces with greatly contrasting styles: a progressive rock piece, and a contrapuntal baroque one, where him and the JAM_BOT were able to trade off each other. See Musical Examples 1[1] and 2[2] for a demonstration of the two pieces with their corresponding training data.

4.2 At every musical gesture (2)

When & How to prompt Another approach to modeling the dual process of consciousness is by attending to *every single input* as it is received. Using this approach, the system listens to each note played by the user, updating its context after a brief delay (e.g., 100 ms to accommodate chord input). This strategy can be used to prompt the model with either harmonic or melodic content. For *melody conditioning*, a longer delay (e.g., 800 ms) allows the system to wait for a sustained note before re-prompting the model. Using this strategy, the system can tightly follow the harmonic or melodic decisions of the performer before seamlessly transitioning to a leading role, until another input is received. To implement a stricter form of conditioning, we use the *anticipatory* mechanism of Anticipatory Music Transformers to repeatedly condition the model on the same prompt every x ms, for a given x. This locks the system into a following role, preventing it from making novel musical decisions.

Training Data & Performance Similarly to the previous case, we can enable this type of interaction by fine-tuning music LMs on long recorded MIDI clips. To model the joint distribution between the harmony/melody conditioning (*conditioning signal*) and the sequence to generate (*input signal*), we create MIDI files with two different instruments–represented by two different General MIDI

[1] `https://jam-bot-ismir-2025.media.mit.edu/#musical-example-1`

[2] `https://jam-bot-ismir-2025.media.mit.edu/#musical-example-2`

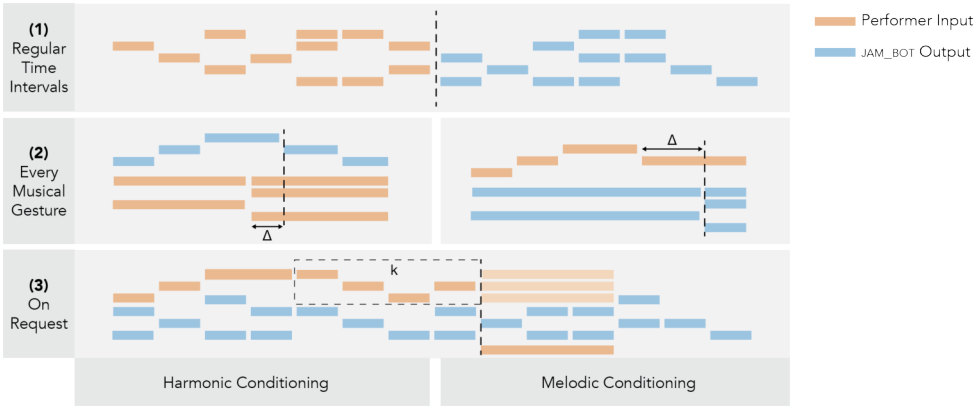


Figure 1. The three different input strategies used to prompt the music LMs–*at regular time intervals*, *at every musical gesture*, and *on request*. Interaction strategies (1) and (3) provide both harmonic and melodic information to the system, while interaction strategy (2) provides either. All provide rhythmic information. Δ refers to the delay parameter set to trigger generation for (2), while k refers to the number of notes to fill the buffer for (3).

codes. These two different instruments are recorded using either using different keyboard registers, or through different MIDI channels. Empirically, we discovered that the best training data was collected when Rudess fixed a conditioning signal (e.g., a specific chord) for a few bars, and improvised multiple input signals over the same condition before transitioning to a new one. In the performance, this type of interaction was used for a *rubato* piece, where Rudess would play sequences of chords, dictating harmonic decisions to the JAM_BOT, which would freely improvise melodies on top. This is exemplified in Musical Example 3.[3] Another example of this interaction strategy is shown in Musical Example 4,[4] where Rudess can prompt the JAM_BOT to offer chord suggestions by playing melodies on top.

4.3 On request (3)

When & How to prompt Finally, we can employ an hybrid approach, which allows the model to alternate explicitly between focus on its own composition and the user input *on request*. Here, the system shifts its attention to the performer's main conditioning input (*conditioning signal 1*) upon request, triggered by a specific input (*conditioning signal 2*–e.g., a specific register of the keyboard). Once activated, the system uses the most recent notes stored in a buffer of predetermined size to condition its future outputs. This approach, for example, can allow the performer to play melodic lines and prompt the system to refresh its output when a root note is played in a lower octave.

Training Data & Performance This type of interaction is slightly more complex than the previous two. In order to fine-tune music LMs and enable this interaction, we must record two different MIDI clips, enabling the modeling of both conditioning signals. In the final performance, Jordan Rudess used this strategy for a piece where the JAM_BOT was able to improvise chords and basslines, while Rudess improvised melodies on top. When he wanted to focus the model's attention on his most recent harmonic and melodic movements, he could hit a root note on the lowest octave of his keyboard to trigger the prompting of the underlying music LM. The training data for this interaction required Rudess to provide two MIDI clips: one with pairs of chords and basslines, and one with pairs of chords and root notes. On stage, Rudess performed with this interaction strategy to create a musical piece where he would improvise melodies over the JAM_BOT's basslines and chord progressions. On cue, Rudess was able to provide a root note to guide the system's harmonic generation. This is exemplified in Musical Example 5.[5]

We note that, for all input strategies, to additionally ensure *local coherence* with the musician's input, we use particularly small context windows (between 40 and 60 notes) to prompt the music LM. This allows the JAM_BOT to generate sequences that connect directly to the performer's input, instead of focusing on musical events that have happened in a more distant past.

5. OPTIMIZING MUSIC LMS FOR REAL-TIME PERFORMANCE

With our interaction strategies designed and custom music LMs fine-tuned, we can now develop a real-time architecture that facilitates seamless integration with the performer. To achieve this, the system must execute the following tasks in parallel:

- Receive MIDI input from the performer, assign timestamps, and convert it into musical events.
- Based on the current *interaction strategy*, aggregate musical events and dynamically create prompts for the music LM.
- Perform autoregressive inference using the current prompt.

[3] https://jam-bot-ismir-2025.media.mit.edu/#musical-example-3

[4] https://jam-bot-ismir-2025.media.mit.edu/#musical-example-4

[5] https://jam-bot-ismir-2025.media.mit.edu/#musical-example-5

- Gather the music LM's output and schedule musical events to be played as MIDI notes.

These tasks must not only run in parallel but also synchronize with a *global clock* to minimize latency. While real-time audio processing typically addresses such challenges, applying these principles to complex Machine Learning algorithms is less common due to inherent latency issues. Additionally, the intricacy and tight coupling of the *interaction strategies* we introduce–compared to simpler tasks like continuation–add complexity to the system design. In this section, we detail the system's architecture and outline the optimizations implemented for each step to ensure efficient performance. We hope that our architecture can inspire future real-time AI-powered music systems.

5.1 Developing a Real-Time Environment

To build a real-time system, we utilize the *JUCE* framework, an open-source, cross-platform C++ framework popular for audio applications [31]. Beyond offering tools for graphical interfaces and MIDI device interaction, it provides numerous thread-safe algorithms and data structures. Based on the four tasks outlined earlier, we partition our system into four parallel threads, each communicating via thread-safe queues (Fig. 2), described below.

Our first thread is the **Clock Thread** which, similarly to Digital Audio Workstations (DAWs), manages continuous synchronization to a global time clock that is either internal, using the machine's clock system, or external, receiving information from another system via MIDI Timecode (MTC). To prevent sudden time jumps when synchronizing with an external clock, we use a *Proportional Control Loop* to update our local time offset δ as follows:

$$\delta_{\text{new}} = \begin{cases} (1-G)\delta_{\text{prev}} + G \cdot \delta & \text{if } |\delta - \delta_{prev}| \leq T \\ \delta & \text{otherwise} \end{cases}$$

where G is a gain parameter, $\delta = (t_{\text{local}} - t_{\text{received}})$ is the new calculated time offset from the time information received via MTC, and T is a threshold parameter.

The **Input Capture Thread** handles MIDI input from the performer and passes it to the **Processing Thread**. If passthrough is enabled, it can also send MIDI input directly to the MIDI synthesizer. The **Processing Thread** timestamps the MIDI notes received by the **Input Capture Thread** using the **Clock Thread**. It then converts these MIDI notes into musical events by combining *Note On* and *Note Off* messages, and creates musical prompts according to the selected *interaction strategy*. These prompts are then passed to the **Generation Thread**. When generation needs to be reset (e.g., after 4 bars when prompting the model at regular time intervals), the **Processing Thread** issues a reset signal to the **Generation Thread**. The **Generation Thread** conducts iterative inferences on the music LM with a dynamically adaptive context, sending generated notes to the **Processing Thread**. If necessary, it can also send meta-signals, such as requests to invalidate previously sent notes. The **Processing Thread** collects the generated MIDI notes from the **Generation Thread** and schedules them for playback, where it sends them to a virtual or hardware MIDI output for synthesis.

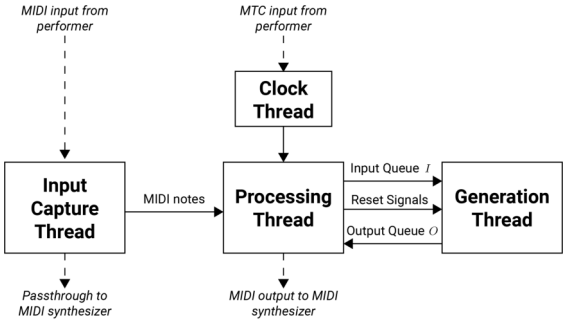


Figure 2. The different parallel processes of the JAM_BOT system, with their interactions.

5.2 Optimizing Deep Neural Networks

To achieve real-time performance, model inference must be highly optimized. Our base model (`stanford-crfm/music-medium-800k`) utilizes the `transformers` Python library, which is built on top of the `PyTorch` framework. Although the model logic can be exported to `torchscript` for execution in C++ via the `PyTorch` C++ API, the generation performance remains suboptimal in this setting. To enhance efficiency, we convert our base model to the *ONNX* framework, enabling optimized execution across various hardware backends. By exporting the model to *ONNX*, we benefit from graph optimizations, operator fusion, and native *CUDA* support through *ONNX Runtime*. This transition allows our system to leverage hardware accelerations beyond what is natively available in *PyTorch*.

We also apply *8-bit weight quantization* to the model using *ONNX Runtime*'s quantization toolkit. This drastically reduces memory footprint and computational cost while maintaining inference accuracy within an acceptable range. Quantization not only reduces the size of the model in memory but also speeds up matrix multiplications, which are the primary bottlenecks in Transformer-based architectures. Finally, we apply *KV caching* to optimize sequential inference. Without caching, each new token generation requires reprocessing the entire input sequence, leading to unnecessary long computations. By storing and reusing attention key-value pairs from previous inference steps, we ensure that subsequent token generations operate with a reduced computational burden, enabling better real-time performances. These optimizations allow us to eventually run inferences on a consumer-grade NVIDIA GeForce RTX 4090 GPU.

6. EVALUATION

6.1 Jordan Rudess's self report

"It's been pretty mind-blowing to create this tech-based version of myself–like looking into a real-time musical mirror. I've played with so many amazing musicians over the years and learned a ton from each experience, but this was something totally different. It gave me a deep, almost ana-

lytical look at how I actually think and play—how musical ideas are built from the rules and instincts I've internalized over time. It's been incredibly educational, not just creatively but in understanding the architecture of my own musical language. And I'm still wide open to where this exploration can go next." (Jordan Rudess)

6.2 Audiences' perception

During the September 2024 performance, we collected audience feedback to assess the perceived impact of the JAM_BOT. Five of our 41 participants were excluded for lack of completion of any portion of the survey, resulting in 36 responses. Participants were asked whether they noticed specific JAM_BOT behaviors during the performance, and most reported observing real-time reactions to the live musicians ($n = 25$) and independent harmonic decision-making ($n = 24$). This suggests that the system effectively follows and leads musical decisions as required. We also assessed whether the output perceptibly strayed from the directions of the performers, defined in this case as "coherence", and found that there was no significant difference between the number of participants who did ($n = 15$) and did not ($n = 13$) feel that the outputs were coherent. This ambiguity suggested that further, more nuanced assessments of these dynamics were needed. Qualitative responses highlighted a range of reactions to the system's behavior and the performance overall, with specific appreciation for the JAM_BOT's responsive, real-time output. However, participants also noted that some of the generated music felt monotonous, while others stated that " AI is going to have a place in the future of music whether someone likes it or not..." and that "it was really amazing how well the model was able to react to the players and create a lovely sound..." These reflections, however, are limited by the context in which they were elicited. Since the concert was open to the public, our sample population varied widely in both musical expertise and AI familiarity. Given the sources of auditory, visual, and social distraction that also characterize live performance contexts, it was especially important to supplement our findings from the performance with controlled evaluations of the efficacy of JAM_BOT. The full description of the study and statistical analyses can be found on our website.[6]

6.3 Comparing JAM_BOT and previous methods

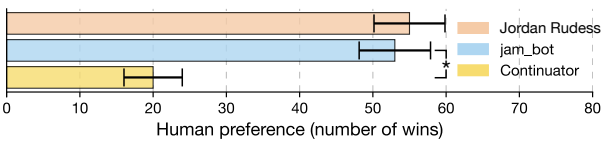


Figure 3. Results depicting number of times a source is preffered in our listening study. Error bars show the standard deviation of a binomial distribution fitted to the binary win/loss counts of each source.

We conduct a listening study ($n = 24$) to baseline a JAM_BOT model that can be prompted at regular time intervals against both Jordan Rudess's playing and Continuator [18], because of how Continuator can be adapted to any stylistic input and the contemporary use of Markov models in real-time contexts [32]. We employ a pairwise comparison in which participants listen to two continuations of the same musical prompt and rate which continuation they prefer for the initial musical prompt on a 5-point Likert scale. The results (Fig. 3) find no significant distinction between the continuations of our method and Jordan Rudess's playing, as well as a strong and significant preference for both of these sources over Continuator. The full description of the study and statistical analyses can be found on our website.[7]

7. CONCLUSION & FUTURE WORK

We introduced the JAM_BOT, a real-time Generative AI system utilizing music Language Models to facilitate collaborative free improvisation with performers. Developed in collaboration with Jordan Rudess, the system was tailored to meet his requirements during a live music performance. This process involved designing three interaction strategies to guide real-time prompting of the music LM, along with selecting and refining the necessary training data. We also developed an optimized system to embed our models and run them in real-time. We hope that our work can foster more collaborations between AI researchers and artists and inspire future development of generative AI music systems for use in live music performances. We believe additional interaction strategies, particularly through the use of Reinforcement Learning, could enhance the system's responsiveness to external musical input. Additionally, we see potential improvements by enabling expressive MIDI outputs and extending the JAM_BOT's capabilities beyond free improvisation to planned improvisation.

"I'm excited about the idea of eventually creating a single, intelligent model that brings together everything we've developed across the separate models so far. Right now, different musical concepts live in different places, but I'd love to see one unified "Jordan Rudess" model that can understand and respond more naturally and fluidly. Even beyond just my own musical input, I see the potential for it to become something even more expansive—capable of drawing from a broader range of influences while still feeling deeply connected to my musical identity." (Jordan Rudess)

8. ACKNOWLEDGMENTS

A special thank you to Jordan and Danielle Rudess, who enthusiastically guided this research project. Thank you to everyone who made this research and performance possible: Lydia Brosnahan, Brian Mayton, Carlo Mandolini, Nathan Perry, Kevin Davis, Cornelle King, Eran Egozy, Audrey Lee, MIT E33, MIT Center for Art Science and Technology, MIT Office of the Arts,

[6] https://jam-bot-ismir-2025.media.mit.edu/#appendix-audience

[7] https://jam-bot-ismir-2025.media.mit.edu/#appendix-listening

and the entire Responsive Environments and MIT Media Lab communities. Thank you to the team at JUCE for developing the framework we used in this project and for providing a new educational license as a result of this work.

9. ETHICS STATEMENT

We developed the JAM_BOT with the intention of *augmenting* the ability of performers and composers, not replacing them, but we must recognize that similar systems have the potential to displace both parties. In an effort to promote artists' rights and autonomy, we used a 1:1 trainer-to-performer system, meaning that JAM_BOT performers own their model and all of its generations. However, future uses of JAM_BOTS are not limited to this approach, which raises concerns about scenarios where performers and composers are seen as completely separate entities from research teams. Without strong connections between the artists and those training the models, we run the risk of jeopardizing artist ownership and creative expression. We also note that the JAM_BOT was pre-trained on predominantly Western music, which limits its musical vocabulary and contributes to broader conversations regarding the dilution of non-Western musical traditions in music generation. Similar issues have been raised by other researchers in the field, though the particular nuance of ethical considerations for real-time systems like JAM_BOTS is worth continued discussion.

10. REFERENCES

[1] L. Blanchard, P. Naseck, E. Egozy, and J. A. Paradiso, "Developing Symbiotic Virtuosity: Ai-Augmented Musical Instruments and Their Use in Live Music Performances," *An MIT Exploration of Generative AI*, sep 25 2024, https://mit-genai.pubpub.org/pub/iz684jjr.

[2] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. Shazeer, I. Simon, C. Hawthorne, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck, "Music Transformer," *arXiv preprint arXiv:1809.04281*, 2018.

[3] J. Thickstun, D. L. W. Hall, C. Donahue, and P. Liang, "Anticipatory music transformer," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=EBNJ33Fcrl

[4] M. Norgaard, "The interplay between conscious and subconscious processes during expert musical improvisation," in *Music and Consciousness 2: Worlds, Practices, Modalities*, R. Herbert, D. Clarke, and E. Clarke, Eds. Oxford University Press, Apr. 2019, p. 0. [Online]. Available: https://doi.org/10.1093/oso/9780198804352.003.0011

[5] T. Funk, "A musical suite composed by an electronic brain: Reexamining the illiac suite and the legacy of lejaren a. hiller jr." *Leonardo Music Journal*, vol. 28, pp. 19–24, 2018.

[6] D. Cope, *The algorithmic composer*. AR Editions, Inc., 2000, vol. 16.

[7] J. Biles *et al.*, "Genjam: A genetic algorithm for generating jazz solos," in *ICMC*, vol. 94. Ann Arbor, MI, 1994, pp. 131–137.

[8] D. Eck and J. Schmidhuber, "A first look at music composition using lstm recurrent neural networks," *Istituto Dalle Molle Di Studi Sull Intelligenza Artificiale*, vol. 103, no. 4, pp. 48–56, 2002.

[9] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, "This time with feeling: Learning expressive musical performance," *Neural Computing and Applications*, vol. 32, pp. 955–967, 2020.

[10] C.-Z. A. Huang, T. Cooijmans, A. Roberts, A. Courville, and D. Eck, "Counterpoint by convolution," in *Proceedings of ISMIR 2017*, 2017.

[11] M. E. Malandro, "Composer's Assistant: An Interactive Transformer for Multi-Track MIDI Infilling," in *Proc. 24th Int. Society for Music Information Retrieval Conf.*, Milan, Italy, 2023, pp. 327–334.

[12] H.-W. Dong, W.-Y. Hsiao, L.-C. Yang, and Y.-H. Yang, "Musegan: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.

[13] H.-W. Dong, K. Chen, S. Dubnov, J. McAuley, and T. Berg-Kirkpatrick, "Multitrack music transformer," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[14] Y.-J. Shih, S.-L. Wu, F. Zalkow, M. Müller, and Y.-H. Yang, "Theme transformer: Symbolic music generation with theme-conditioned transformer," *IEEE Transactions on Multimedia*, vol. 25, pp. 3495–3508, 2022.

[15] S.-L. Wu and Y.-H. Yang, "Musemorphose: Full-song and fine-grained piano music style transfer with one transformer vae," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1953–1967, 2023.

[16] B. L. Vercoe, "Synthetic listeners and synthetic performers," *The Journal of the Acoustical Society of America*, vol. 88, no. S1, pp. S70–S70, 1990.

[17] G. E. Lewis, "Too many notes: Computers, complexity and culture in voyager," *Leonardo music journal*, vol. 10, pp. 33–39, 2000.

[18] F. Pachet, "The continuator: Musical interaction with style," *Journal of New Music Research*, vol. 32, no. 3, pp. 333–341, 2003.

[19] G. Assayag, G. Bloch, M. Chemillier, A. Cont, and S. Dubnov, "Omax brothers: a dynamic yopology of agents for improvization learning," in *Proceedings of the 1st ACM workshop on Audio and music computing multimedia*, 2006.

[20] G. Weinberg and S. Driscoll, "Robot-human interaction with an anthropomorphic percussionist," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ser. CHI '06. New York, NY, USA: Association for Computing Machinery, 2006, p. 1229–1232. [Online]. Available: https://doi.org/10.1145/1124772.1124957

[21] G. Weinberg, M. Godfrey, A. Rae, and J. Rhoads, "A real-time genetic algorithm in human-robot musical improvisation," in *Computer Music Modeling and Retrieval. Sense of Sounds*, R. Kronland-Martinet, S. Ystad, and K. Jensen, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 351–359.

[22] G. Hoffman and G. Weinberg, "Gesture-based human-robot jazz improvisation," in *2010 IEEE international conference on robotics and automation*. IEEE, 2010, pp. 582–587.

[23] A. Caillon and P. Esling, "Rave: A variational autoencoder for fast and high-quality neural audio synthesis," 2021. [Online]. Available: https://arxiv.org/abs/2111.05011

[24] S. Nabi, P. Esling, G. Peeters, and F. Bevilacqua, "Embodied exploration of deep latent spaces in interactive dance-music performance," in *Proceedings of the 9th International Conference on Movement and Computing*, ser. MOCO '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3658852.3659072

[25] C. Benetatos, J. VanderStel, and Z. Duan, "Bachduet: A deep learning system for human-machine counterpoint improvisation," in *Proceedings of the International Conference on New Interfaces for Musical Expression*, 2020.

[26] C. Donahue, I. Simon, and S. Dieleman, "Piano genie," in *Proceedings of the 24th International Conference on Intelligent User Interfaces*, 2019, pp. 160–164.

[27] Y. Wu, T. Cooijmans, K. Kastner, A. Roberts, I. Simon, A. Scarlatos, C. Donahue, C. Tarakajian, S. Omidshafiei, A. Courville, P. S. Castro, N. Jaques, and C.-Z. A. Huang, "Adaptive accompaniment with realchords," in *International Conference on Machine Learning*, 2024.

[28] A. Scarlatos, Y. Wu, I. Simon, A. Roberts, T. Cooijmans, N. Jaques, C. Tarakajian, and C.-Z. A. Huang, "Realjam: Real-time human-ai music jamming with reinforcement learning-tuned transformers," *CHI Late-Breaking Works track*, 2025.

[29] C. Raffel, *Learning-based methods for comparing sequences, with applications to audio-to-midi alignment and matching*. Columbia University, 2016.

[30] Vigliensoni, Gabriel, Perry, Phoenix, and Fiebrink, Rebecca, "A Small-Data Mindset for Generative AI Creative Work," New York, NY, USA, May 2022. [Online]. Available: https://ualresearchonline.arts.ac.uk/id/eprint/18343/1/CHI_Workshop_GenAI.pdf

[31] J. Storer, "Juce," https://github.com/juce-framework/JUCE, 2004.

[32] O. Ben-Tal and D. Dolan, "Musical and meta-musical conversations," 2023.